

Notebook 3: Shift-WING CaseB 200epoch実行結果

Notebook handout

Source notebook: Transolver_Learn/japanese_notebooks/03_training_transolver_shiftwing_caseb_ja_full200_executed.ipynb

このPDFはNotebook本文、標準出力、matplotlib図を含む確認用資料です。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 1

Notebook 3: Shift-WINGでTransolverをスクラッチ学習する

Notebook 1/2では、Transolverがなぜ大規模な点群・メッシュデータに向いているのか、Physics-Attentionがどのように点群を少数の物理tokenへ集約するのかを見ました。

このNotebookの目的は、単にコマンドを実行することではありません。学習者が、Shift-WINGのCFDデータからTransolver surrogate modelを作るときに、次の一連の流れを自分で説明できるようになることです。

1. 何を入力し、何を予測する問題なのかを定義する。
2. train / validation / testを分け、未知形状への汎化を評価できる状態にする。
3. CFD表面データを、Transolverに入る点群テンソルへ変換する。
4. 事前学習なしのTransolverを、どのアーキテクチャと学習条件で訓練するか決める。
5. 学習曲線から、学習が進んだのか、過学習していないかを判断する。
6. best modelをtest splitで評価し、Pressure誤差を数値と図の両方で読む。

今回の題材は、過去資料の Case B: Transolver scratch に合わせます。GeoPTの事前学習済み重みは使わず、Transolverをランダム初期化から学習します。一方で、比較条件を崩さないため、Transolverのアーキテクチャや主要ハイパーパラメータは過去CaseBと同じにします。

このNotebookで最終的に確認する結果

この環境の200 epoch runでは、validation relative L2のbestはおよそ 0.00848、best epochは 178 でした。test splitのPressure誤差は、点ごとの誤差割合で中央値が約 0.22%、P90が約 0.98% です。後半のセルでは、この結果を学習曲線、誤差分布、コンター風図として確認します。

用語メモ

- scratch学習: 事前学習済み重みをロードせず、ランダム初期化から学習すること。
- surrogate model: CFDなど重いシミュレーションの代わりに、近似的に物理量を予測するAIモデル。
- validation: 学習中に性能確認へ使うデータ。重み更新には使わない。
- test: 学習完了後に最終評価へ使うデータ。モデル選択にも使わない。
- relative L2: 予測誤差のL2ノルムを真値のL2ノルムで割った指標。小さいほどよい。
- normalization: 目的変数の平均・標準偏差でスケールを揃え、学習を安定させる処理。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 2

1. 今回の学習シナリオ

元のOpenHackathons Notebook3は、Ahmed Body datasetとPhysicsNeMo containerを前提にしたTransolver学習教材です。ここでは題材をShift-WINGへ置き換え、ローカルにあるCaseB実験の設定でスクラッチ学習を再現します。

観点	元Notebook3	今回版
---	---	---
物理対象	Ahmed Body周りの流れ	Shift-WING翼面周りの流れ
データ	Ahmed Body vehicle dataset	OnShape_luminary_crm_version001
入力	車体表面メッシュと条件	翼面点群、法線、Mach/AoA/beta
出力	表面上の物理量	Pressure と Wall Shear Stress 3成分
モデル	PhysicsNeMo Transolver	GeoPT実験で使った公式Transolver実装
学習方針	demo training	CaseB scratchの200 epoch学習

学習者として注目すべき点は、「題材が変わっても、点群surrogateの基本手順は同じ」ということです。物理対象がAhmed Bodyから翼へ変わっても、データを点群テンソルにし、条件を特徴として加え、点ごとの物理量を回帰する、という流れは共通です。

今回使うデータと前処理済み配列は以下です。

```
```text
EXPERIMENT/GeoPT/data/Wing-sample/OnShape_luminary_crm_version001
EXPERIMENT/GeoPT/preprocessed/wing_geopt_npys
```
```

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 3

<!-- transolver-learning-note -->

学習の準備: 実験を再現できる状態にする

このセルでは、学習そのものではなく、実験を再現するための土台を作ります。重要なのは、データ、設定ファイル、GeoPT側のhelper関数、Transolver実装を同じ場所から読むことです。

AIの学習実験では、モデルだけでなく「どの前処理」「どのsplit」「どのnormalization」「どのスクリプト」を使ったかが結果を左右します。このセルは、その依存関係を明示して、以降のセルが同じ条件で動くようにしています。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 1

```
# --- Pythonの型ヒントを扱いやすくする設定です。 ---
from __future__ import annotations

# --- 設定ファイル、パス操作、外部コマンド実行に必要な標準ライブラリです。 ---
import json
import os
import subprocess
import sys
from pathlib import Path

# --- 数値計算、描画、PyTorchモデル実行に必要なライブラリです。 ---
import matplotlib.pyplot as plt
import numpy as np
import torch

# --- どこからNotebookを実行してもrepo rootを見つけるための関数です。 ---
def find_repo_root(start: Path | None = None) -> Path:
    start = (start or Path.cwd()).resolve()
    candidates = [start, *start.parents]
    for candidate in candidates:
        if (candidate / "EXPERIMENT" / "GeoPT").exists():
            return candidate
    raise RuntimeError("Could not find repository root containing EXPERIMENT/GeoPT")

# --- 以降で使う主要ディレクトリをまとめて定義します。 ---
REPO_ROOT = find_repo_root()
GEOPT_ROOT = REPO_ROOT / "EXPERIMENT" / "GeoPT"
SCRIPTS_DIR = GEOPT_ROOT / "scripts"

# --- GeoPT側のhelper scriptをimportできるよう、Pythonの探索パスへ追加します。 ---
```

```
if str(SCRIPTS_DIR) not in sys.path:
    sys.path.insert(0, str(SCRIPTS_DIR))
```

```
# --- 学習済み実験と同じ前処理・モデル構築helperを読み込みます。 ---
```

```
from wing_geopt_common import ( # noqa: E402
```

```
    build_geopt_model_args,
    import_geopt_transolver,
    load_preprocessed_sample,
    read_json,
    resolve_device,
    torch_craft_direction,
)
```

```
# --- Shift-WINGデータ、split定義、前処理済み配列の場所を指定します。 ---
```

```
DATA_ROOT = GEOPT_ROOT / "data" / "Wing-sample" / "OnShape_luminary_crm_version001"
```

```
SPLIT_PATH = GEOPT_ROOT / "configs" / "wing_split_pca_extreme.json"
```

```
PREPROCESSED_DIR = GEOPT_ROOT / "preprocessed" / "wing_geopt_npys"
```

```
RUNBOOK_PATH = GEOPT_ROOT / "RUNBOOK.md"
```

```
print("REPO_ROOT:", REPO_ROOT)
```

```
print("DATA_ROOT exists:", DATA_ROOT.exists())
```

```
print("SPLIT_PATH exists:", SPLIT_PATH.exists())
```

```
print("PREPROCESSED_DIR exists:", PREPROCESSED_DIR.exists())
```

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 1 output

REPO_ROOT: /home/hiroaki-ozasa/Desktop/Development/PINN_CFD

DATA_ROOT exists: True

SPLIT_PATH exists: True

PREPROCESSED_DIR exists: True

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 5

2. 予測問題を定義する

最初に決めるべきことは、「AIに何を入力し、何を出力させるか」です。今回のsurrogate modelは、1つの翼形状・流入条件に対して、翼表面上の各点の物理量を予測します。

入力側には、各表面点の座標、表面法線、ケース条件を使います。ケース条件は Mach、AoA、beta です。出力側は点ごとの4成分です。

```
```text
y = [Pressure, Wall Shear Stress x, Wall Shear Stress y, Wall Shear Stress z]
```
```

各出力の意味は次の通りです。

| 出力 | 意味 |
|---------------------|-------------------------------|
| Pressure | 翼表面上の圧力。単位は元CFDデータの圧力単位に従います。 |
| Wall Shear Stress x | 壁面せん断応力ベクトルのx成分。 |
| Wall Shear Stress y | 壁面せん断応力ベクトルのy成分。 |
| Wall Shear Stress z | 壁面せん断応力ベクトルのz成分。 |

Wall Shear Stressは、流体が壁面をこすることによって生じる接線方向の応力です。表面法線とは別物です。表面法線は幾何形状を表す入力特徴であり、Wall Shear StressはCFDで得られる予測対象の物理量です。

つまりこれは、画像分類のように1サンプルへ1ラベルを出す問題ではありません。1つの翼サンプルに対して、数十万から百万点規模の各点に4成分の値を出す 点ごとの回帰問題 です。

各sampleディレクトリには、主に次のファイルがあります。

| File | 内容 |

|---|---|

| merged_surfaces.vtp | 翼表面メッシュとCFD結果の点データ |

| params.json | Mach数、迎角などの条件・形状パラメータ |

| forces.json | 力や係数に関する値 |

| metadata.json | sampleのメタ情報 |

このNotebookでは、重いVTPを毎回読むのではなく、既に作成済みのNumPy配列を使って学習の流れを追います。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 6

<!-- transolver-learning-note -->

学習の準備: データ分割を確認する

このセルでは、train、validation、testに含まれるサンプル名を確認します。trainは重み更新、validationは学習中のモデル選択、testは最後の未知形状評価に使います。

ここを曖昧にすると、学習に使った形状で評価してしまい、surrogate modelが本当に未知形状へ使えるのか判断できなくなります。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 2

```
# --- train/validation/testのサンプル名リストを読み込みます。 ---
```

```
split = read_json(SPLIT_PATH)
```

```
manifest = read_json(PREPROCESSED_DIR / "manifest.json")
```

```
# --- 各splitの件数を確認し、意図した分割になっているか見ます。 ---
```

```
print("Split counts:")
```

```
for key in ["train", "val", "test"]:
```

```
    print(f" {key}: {len(split[key])}")
```

```
print("\nFirst train samples:", split["train"][:5])
```

```
print("Validation samples:", split["val"])
```

```
print("Test samples:", split["test"])
```

```
print("\nPreprocess target_len:", manifest.get("target_len"))
```

```
print("Preprocessed records:", len(manifest.get("records", [])))
```

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 2 output

Split counts:

train: 80

val: 10

test: 10

First train samples: ['sample_000002', 'sample_000003', 'sample_000004', 'sample_000006', 'sample_000007']

Validation samples: ['sample_000009', 'sample_000011', 'sample_000012', 'sample_000018', 'sample_000033', 'sample_000053']

['sample_000070', 'sample_000082', 'sample_000086', 'sample_000096']

Test samples: ['sample_000001', 'sample_000005', 'sample_000010', 'sample_000029', 'sample_000043', 'sample_000047', 'sample_000066', 'sample_000069', 'sample_000073', 'sample_000095']

Preprocess target_len: 5.0

Preprocessed records: 100

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 8

3. データがモデルへ入る形を理解する

CFDの元データはVTPですが、ニューラルネットへ入れるときはテンソルにする必要があります。過去実験では、VTPから学習しやすい.npy 配列を作成済みです。

各sampleには次の3種類があります。

```
```text
x_sample_*.npy shape = (N, 7)
y_sample_*.npy shape = (N, 4)
cond_sample_*.npy shape = (3,)
```
```

N はそのサンプルの表面点数です。x の7次元は次の通りです。

| Index | Feature | 内容 |
|-------|--------------|--------------------------------|
| 0 | x | 正規化済み座標x |
| 1 | y | 正規化済み座標y |
| 2 | z | 正規化済み座標z |
| 3 | surface flag | 今回は0固定。点が表面点であることを表すためのフラグ枠です。 |
| 4 | normal_x | 表面法線ベクトルのx成分 |
| 5 | normal_y | 表面法線ベクトルのy成分 |
| 6 | normal_z | 表面法線ベクトルのz成分 |

表面法線は速度ではありません。翼表面の各点で「表面の外側に垂直に立つ向き」を表す幾何ベクトルです。例えば、表面の向きが上を向いている点、横を向いている点、下を向いている点では、同じ座標付近でも流れに対する受け方が変わります。Transolverへ法線を入れることで、モデルは表面の向きも手がかりにできます。

cond は、sample全体に共通する流入条件です。

```
```text
cond = [Mach, AoA, beta]
```
```

| | |
|------|-------------------------------|
| 条件 | 意味 |
| --- | --- |
| Mach | マッハ数。流速を音速で割った無次元量。 |
| AoA | Angle of Attack、迎角。翼に対する流入角度。 |
| beta | 横滑り角、または横方向の流入角度。 |

この cond はそのまま点ごとの特徴にはなっていないため、後続のforward passで4次元の流入条件promptへ変換します。

```
```text
flow-condition prompt = [flow_dir_x, flow_dir_y, flow_dir_z, Mach / 3]
```
```

flow_dir_x, flow_dir_y, flow_dir_z は、AoA と beta から作る流入方向ベクトルです。Mach / 3 は Mach数を大きすぎないスケールにした追加特徴です。この4次元を全ての点に同じ値として付与し、各点の7次元特徴と連結します。

最終的にTransolverへ渡す点特徴は次の11次元になります。

```
```text
fx = [x, y, z, surface_flag, normal_x, normal_y, normal_z,
 flow_dir_x, flow_dir_y, flow_dir_z, Mach/3]
```
```

ここで重要なのは、Transolverが「点の並び順」ではなく「点ごとの座標・特徴」として物理場を扱うことです。点群は規則格子でなくてもよく、各点に座標と特徴があれば学習できます。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 9

<!-- transolver-learning-note -->

学習の準備: 1サンプルのshapeを読む

ここでは、1つのtrain sampleを読み、点群入力 x 、教師値 y 、条件 $cond$ のshapeを確認します。学習者は、以降のモデル設定がこのshapeから決まっていることを意識してください。

特に、 y が4次元なので $out_dim=4$ 、 x と条件promptを合わせて $fun_dim=11$ になります。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 3

```
# --- train splitの先頭サンプルを例として読み込みます。 ---
sample_name = split["train"][0]

# --- 前処理済みのx, y, condをまとめて読み込みます。 ---
sample = load_preprocessed_sample(PREPROCESSED_DIR, sample_name)

# --- 入力、教師、条件のshapeを出力し、モデル設定と合うか確認します。 ---
print("Sample:", sample_name)
print("x shape:", sample["x"].shape)
print("y shape:", sample["y"].shape)
print("cond shape:", sample["cond"].shape)
print("cond [Mach, AoA, beta]:", sample["cond"])

for i, name in enumerate(["pressure", "wss_x", "wss_y", "wss_z"]):
    values = sample["y"][ :, i]
    print(f"{name:8}: min={values.min(): .4e}, mean={values.mean(): .4e}, max={values.max(): .4e}")
```

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 3 output

Sample: sample_000002

x shape: (1647276, 7)

y shape: (1647276, 4)

cond shape: (3,)

cond [Mach, AoA, beta]: [0.5 3.3782 0.]

pressure: min= 1.9177e+04, mean= 2.3472e+04, max= 2.8300e+04

wss_x : min=-9.9637e+00, mean= 9.9716e+00, max= 5.0180e+01

wss_y : min=-2.4295e+01, mean=-2.9321e-04, max= 2.4410e+01

wss_z : min=-9.5095e+00, mean= 8.1093e-01, max= 3.5777e+01

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 11

4. 学習前にデータを目で確認する

モデルを作る前に、まず1サンプルの物理量分布を見ます。これは単なる可視化ではなく、学習失敗を防ぐための確認です。

確認したいことは次です。

- 座標の向きやスケールが想定通りか。
- Pressureの高低が翼面上で連続的に分布しているか。
- Wall Shear Stressが極端な外れ値だらけになっていないか。
- 点数が多すぎるため、学習時にサンプリングが必要であることを実感する。

ここではPDFで見やすいように、3Dレンダリングではなく x-y 平面へのscatter表示を使います。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 12

<!-- transolver-learning-note -->

学習の準備: 物理量分布の妥当性を確認する

このセルは、モデルを良くするための直接的な処理ではありません。しかし、学習前のデータ可視化は重要です。PressureやWSSの分布が不自然なら、どれだけモデルやepoch数を調整しても良い結果にはなりません。

点数が多いため、ここでは一部の点だけを抜き出して分布を見ます。これは学習時のpoint samplingと同じ考え方です。

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 4

--- 全点を描くと重いため、可視化用に一部の点だけ選びます。 ---

```
rng = np.random.default_rng(42)
```

```
num_plot = min(25000, sample["x"].shape[0])
```

```
ids = rng.choice(sample["x"].shape[0], size=num_plot, replace=False)
```

```
points = sample["x"][ids, :3]
```

```
pressure = sample["y"][ids, 0]
```

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(points[:, 0], points[:, 1], c=pressure, s=1, cmap="coolwarm")
```

```
plt.xlabel("normalized x")
```

```
plt.ylabel("normalized y")
```

```
plt.title(f"Shift-WING sample pressure: {sample_name}")
```

```
plt.colorbar(label="Pressure [Pa]")
```

```
plt.tight_layout()
```

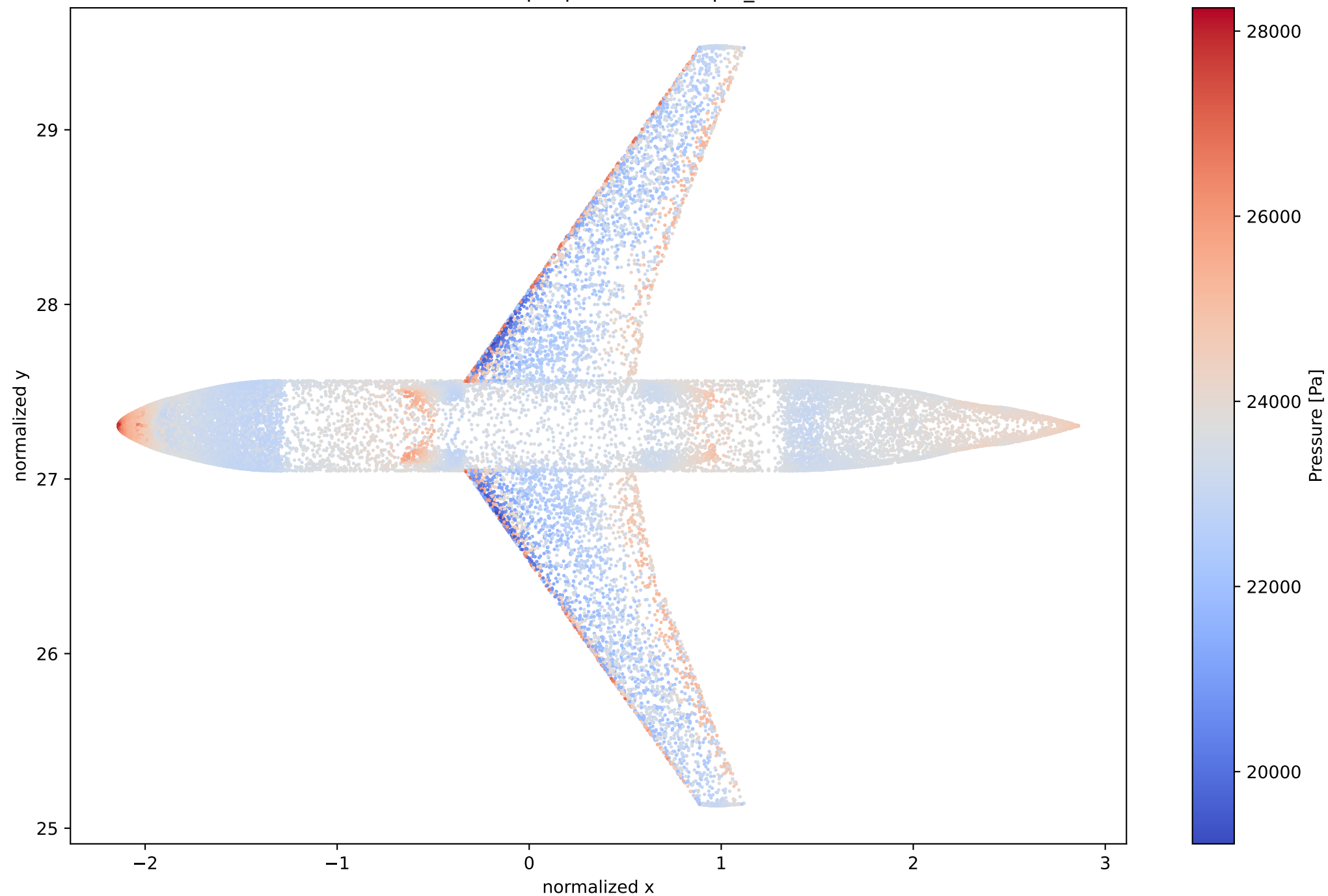
```
plt.show()
```

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 4 output

03_training_transolver_shiftwing_caseb_ja_full200_executed.ipynb:cell13:15: UserWarning: FigureCanvasAgg is not interactive, and thus cannot be shown

Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 4 - Figure 1

Shift-WING sample pressure: sample_000002



Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 14

5. Transolverアーキテクチャを決める

ここで、学習に使うモデルを作ります。今回の目的は「CaseB scratchを教材として再現する」ことなので、モデル構造は過去実験から変更しません。変えるのは学習済み重みを使うかどうかだけです。

CaseBでは、GeoPT事前学習重みを読み込まず、Transolverをランダム初期化します。設定は次の通りです。

| Item | Value | 意味 |
|-----------|-------|--------------------------------------|
| fun_dim | 11 | 点ごとの入力特徴次元。7次元の点特徴 + 4次元の流入条件prompt。 |
| space_dim | 3 | 座標次元。x[:, :, :3] としてモデルへ渡す。 |
| out_dim | 4 | Pressure + Wall Shear Stress 3成分。 |
| n_hidden | 256 | 内部特徴の幅 |
| n_layers | 8 | Transolver blockの層数 |
| n_heads | 8 | token attentionのhead数 |
| mlp_ratio | 2 | block内MLPの幅倍率 |
| slice_num | 32 | physics slice数 |
| dropout | 0.0 | 今回はdropoutなし |

slice_num=32 がTransolverらしい部分です。全点どうしを直接Attentionする代わりに、点群を32個のphysics sliceへsoftに集約し、そのslice token間で相互作用を学習します。

fun_dim=11 の内訳は次です。

```
```text
```

```
7 point features = [x, y, z, surface_flag, normal_x, normal_y, normal_z]
```

```
4 prompt features = [flow_dir_x, flow_dir_y, flow_dir_z, Mach/3]
```

```
7 + 4 = 11
```

```
...
```

space\_dim=3 と fun\_dim=11 が両方ある点に注意してください。GeoPT Transolver実装では、座標  $x[: , :, :3]$  を空間情報として渡し、さらに  $fx$  として11次元の点特徴も渡します。内部では座標3次元と点特徴11次元を連結してembeddingします。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 15

<!-- transolver-learning-note -->

## モデル設計: なぜこのTransolver設定で学習するのか

このセルでは、CaseBと同じTransolverを作ります。ここでの学習上の判断は、「アーキテクチャを変えず、事前学習重みだけを使わない」という点です。

これにより、GeoPT事前学習あり/なしの比較や、別題材への応用時に、モデル構造の違いではなく学習条件の違いとして結果を読めます。slice\_num=32 は、点群を32個のphysics tokenへ集約する設定で、Transolverの計算効率と表現力のバランスを決める重要な値です。

出力されるparameter countは、この設定のモデル規模です。今回は小型化した教材モデルではなく、CaseBで使った設定そのものを確認します。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 5

```
--- モデルの学習対象パラメータ数を数える補助関数です。 ---
def count_parameters(model: torch.nn.Module) -> int:
 return sum(p.numel() for p in model.parameters())

--- CaseBと同じTransolverアーキテクチャ設定を作ります。 ---
model_args = build_geopt_model_args(
 out_dim=4,
 n_hidden=256,
 n_layers=8,
 n_heads=8,
 mlp_ratio=2,
 slice_num=32,
 dropout=0.0,
 checkpoint=0,
)

--- GeoPT実験で使ったTransolverクラスを読み込みます。 ---
model_cls = import_geopt_transolver()

--- 事前学習重みをロードせず、ランダム初期化モデルを作ります。 ---
model = model_cls(model_args)

--- モデル設定とパラメータ数を出力し、意図した規模が確認します。 ---
print(model_args)
print("Parameter count:", f"{count_parameters(model):,}")
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 5 output

```
/home/hiroaki-ozasa/Desktop/Development/PINN_CFD/.venv/lib/python3.12/site-packages/timm/models/layers/__init__
_.py:49: F
utureWarning: Importing from timm.models.layers is deprecated, please import via timm.layers
 warnings.warn(f'Importing from {__name__} is deprecated, please import via timm.layers', FutureWarning)
namespace(fun_dim=11, space_dim=3, out_dim=4, n_hidden=256, n_layers=8, n_heads=8, mlp_ratio=2, slice_num=32,
dropout=0.
0, checkpoint=0, geotype='unstructured', shapelist=None, unified_pos=False, act='gelu')
Parameter count: 3,864,388
```

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 17

## 6. 参考: Transolverアーキテクチャの実装を見る

ここまでで作った model は、Notebook内に直接クラス定義を書いているわけではありません。次の流れで、GeoPT vendor側の実装を読み込んでいます。

```
```text
wing_geopt_common.import_geopt_transolver()
-> EXPERIMENT/GeoPT/vendor/GeoPT/models/Transolver.py の Model
-> EXPERIMENT/GeoPT/vendor/GeoPT/layers/Physics_Attention.py の Physics_Attention_Irregular_Mesh
```
```

この節では、学習に使っている実装の主要部分を参考として表示します。全部を暗記する必要はありません。見るべき対応関係は次です。

|                  |                                                      |                                      |
|------------------|------------------------------------------------------|--------------------------------------|
| 理論での名前           | 実装上の場所                                               | 何をしているか                              |
| --- --- ---      |                                                      |                                      |
| 入力embedding      | Model.preprocess                                     | 座標と点特徴をhidden次元へ変換する                 |
| Transolver block | Transolver_block                                     | Physics-Attention + MLP + residual接続 |
| Slice            | Physics_Attention_Irregular_Mesh.forward の (1) Slice | 点をphysics sliceへsoft as signmentする   |
| Token Attention  | (2) Attention among slice tokens                     | slice token間でAttentionする             |
| Deslice          | (3) Deslice                                          | 更新されたslice tokenを各点へ戻す               |

このコードは参照用です。このNotebookではアーキテクチャを変更せず、CaseBと同じ実装をそのまま使います。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 6

```
--- 参照用: Notebook3で実際に呼んでいるTransolver実装の主要クラスを表示します。 ---
```

```
import inspect
```

```
from pathlib import Path
```

```
--- import_geopt_transolver() を呼ぶと、GeoPT vendor pathがsys.pathへ追加されます。 ---
```

```
_ = import_geopt_transolver()
```

```
--- ここから先は、実際のモデル定義ファイルからクラスをimportします。 ---
```

```
from models.Transolver import Model as GeoPTTransolverModel, Transolver_block # noqa: E402
```

```
from layers.Physics_Attention import Physics_Attention_Irregular_Mesh # noqa: E402
```

```
--- 長すぎる出力にならないよう、主要クラスごとに区切って表示します。 ---
```

```
def show_source(title: str, obj):
```

```
 print()
```

```
 print("=" * 88)
```

```
 print(title)
```

```
 print("=" * 88)
```

```
 print(inspect.getsource(obj))
```

```
--- モデル全体: 入力embedding、Transolver blockの積み重ね、forwardの流れを見ます。 ---
```

```
show_source("models/Transolver.py: class Model", GeoPTTransolverModel)
```

```
--- 1層分のblock: Physics-Attention、MLP、残差接続の構造を見ます。 ---
```

```
show_source("models/Transolver.py: class Transolver_block", Transolver_block)
```

```
--- Physics-Attention本体: Slice、slice token間Attention、Desliceの実装を見ます。 ---
```

```
show_source("layers/Physics_Attention.py: class Physics_Attention_Irregular_Mesh", Physics_Attention_Irregular_Mesh)
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 6 output

```
models/Transolver.py: class Model
```

```
class Model(nn.Module):
 def __init__(self, args):
 super(Model, self).__init__()
 self.__name__ = 'Transolver'
 self.args = args
 ## embedding
 self.preprocess = MLP(args.fun_dim + args.space_dim, args.n_hidden * 2, args.n_hidden,
 n_layers=0, res=False, act=args.act)

 ## models
 self.blocks = nn.ModuleList([Transolver_block(num_heads=args.n_heads, hidden_dim=args.n_hidden,
 dropout=args.dropout,
 act=args.act,
 mlp_ratio=args.mlp_ratio,
 out_dim=args.out_dim,
 slice_num=args.slice_num,
 last_layer=(_ == args.n_layers - 1),
 geotype=args.geotype,
 shapelist=args.shapelist)
 for _ in range(args.n_layers)])

 self.placeholder = nn.Parameter((1 / (args.n_hidden)) * torch.rand(args.n_hidden, dtype=torch.float))
 self.initialize_weights()

 def initialize_weights(self):
 self.apply(self._init_weights)
```

```

def _init_weights(self, m):
 if isinstance(m, nn.Linear):
 trunc_normal_(m.weight, std=0.02)
 if isinstance(m, nn.Linear) and m.bias is not None:
 nn.init.constant_(m.bias, 0)
 elif isinstance(m, (nn.LayerNorm, nn.BatchNorm1d)):
 nn.init.constant_(m.bias, 0)
 nn.init.constant_(m.weight, 1.0)

```

```

def structured_geo(self, x, fx):
 if self.args.unified_pos:
 x = self.pos.repeat(x.shape[0], 1, 1)
 if fx is not None:
 fx = torch.cat((x, fx), -1)
 fx = self.preprocess(fx)
 else:
 fx = self.preprocess(x)
 fx = fx + self.placeholder[None, None, :]

 for block in self.blocks:
 if self.args.checkpoint:
 fx = checkpoint.checkpoint(block, fx)
 else:
 fx = block(fx)
 return fx

```

```

def unstructured_geo(self, x, fx):
 if fx is not None:
 fx = torch.cat((x, fx), -1)
 fx = self.preprocess(fx)
 else:
 fx = self.preprocess(x)

```

```
fx = fx + self.placeholder[None, None, :]
```

```
for block in self.blocks:
```

```
 if self.args.checkpoint:
```

```
 fx = checkpoint.checkpoint(block, fx)
```

```
 else:
```

```
 fx = block(fx)
```

```
return fx
```

```
def forward(self, x, fx):
```

```
 if self.args.geotype == 'unstructured':
```

```
 return self.unstructured_geo(x, fx)
```

```
 else:
```

```
 return self.structured_geo(x, fx)
```

```
=====
```

```
models/Transolver.py: class Transolver_block
```

```
=====
```

```
class Transolver_block(nn.Module):
```

```
 """Transolver encoder block."""
```

```
 def __init__(
```

```
 self,
```

```
 num_heads: int,
```

```
 hidden_dim: int,
```

```
 dropout: float,
```

```
 act='gelu',
```

```
 mlp_ratio=4,
```

```
 last_layer=False,
```

```
 out_dim=1,
```

```
 slice_num=32,
```

```

 geotype='unstructured',
 shapelist=None
):
 super().__init__()
 self.last_layer = last_layer
 self.ln_1 = nn.LayerNorm(hidden_dim)

 self.Attn = PHYSICS_ATTENTION[geotype](hidden_dim, heads=num_heads, dim_head=hidden_dim // num_heads,
 dropout=dropout, slice_num=slice_num, shapelist=shapelist)
 self.ln_2 = nn.LayerNorm(hidden_dim)
 self.mlp = MLP(hidden_dim, hidden_dim * mlp_ratio, hidden_dim, n_layers=0, res=False, act=act)
 if self.last_layer:
 self.ln_3 = nn.LayerNorm(hidden_dim)
 self.mlp2 = nn.Linear(hidden_dim, out_dim)

 def forward(self, fx):
 fx = self.Attn(self.ln_1(fx)) + fx
 fx = self.mlp(self.ln_2(fx)) + fx
 if self.last_layer:
 return self.mlp2(self.ln_3(fx))
 else:
 return fx

```

```

=====
layers/Physics_Attention.py: class Physics_Attention_Irregular_Mesh
=====

```

```

class Physics_Attention_Irregular_Mesh(nn.Module):
 ## for irregular meshes in 1D, 2D or 3D space
 def __init__(self, dim, heads=8, dim_head=64, dropout=0., slice_num=64, shapelist=None):
 super().__init__()
 inner_dim = dim_head * heads

```

```

self.dim_head = dim_head
self.heads = heads
self.scale = dim_head ** -0.5
self.softmax = nn.Softmax(dim=-1)
self.dropout = nn.Dropout(dropout)
self.temperature = nn.Parameter(torch.ones([1, heads, 1, 1]) * 0.5)

self.in_project_x = nn.Linear(dim, inner_dim)
self.in_project_fx = nn.Linear(dim, inner_dim)
self.in_project_slice = nn.Linear(dim_head, slice_num)
for l in [self.in_project_slice]:
 torch.nn.init.orthogonal_(l.weight) # use a principled initialization
self.to_q = nn.Linear(dim_head, dim_head, bias=False)
self.to_k = nn.Linear(dim_head, dim_head, bias=False)
self.to_v = nn.Linear(dim_head, dim_head, bias=False)
self.to_out = nn.Sequential(
 nn.Linear(inner_dim, dim),
 nn.Dropout(dropout)
)

def forward(self, x, return_feature=False, vis=False):
 # B N C
 B, N, C = x.shape

 ### (1) Slice
 fx_mid = self.in_project_fx(x).reshape(B, N, self.heads, self.dim_head) \
 .permute(0, 2, 1, 3).contiguous() # B H N C
 x_mid = self.in_project_x(x).reshape(B, N, self.heads, self.dim_head) \
 .permute(0, 2, 1, 3).contiguous() # B H N C
 slice_weights = self.softmax(self.in_project_slice(x_mid) / self.temperature) # B H N G
 if vis:
 np.save("slice_weights.npy", slice_weights.detach().cpu().numpy())

```

```
slice_norm = slice_weights.sum(2) # B H G
slice_token = torch.einsum("bhnc,bhng->bhgc", fx_mid, slice_weights)
slice_token = slice_token / ((slice_norm + 1e-5)[:, :, None].repeat(1, 1, 1, self.dim_head))
```

```
(2) Attention among slice tokens
```

```
q_slice_token = self.to_q(slice_token)
k_slice_token = self.to_k(slice_token)
v_slice_token = self.to_v(slice_token)
dots = torch.matmul(q_slice_token, k_slice_token.transpose(-1, -2)) * self.scale
attn = self.softmax(dots)
attn = self.dropout(attn)
out_slice_token = torch.matmul(attn, v_slice_token) # B H G D
```

```
(3) Deslice
```

```
out_x = torch.einsum("bhgc,bhng->bhnc", out_slice_token, slice_weights)
out_x = rearrange(out_x, 'b h n d -> b n (h d)')
if return_feature:
 return self.to_out(out_x), slice_token
else:
 return self.to_out(out_x)
```

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 19

## 7. 学習前にforward passを確認する

モデルを学習する前に、1回だけ推論方向へデータを流してshapeを確認します。この段階では重みはランダムなので、予測値そのものに物理的意味はありません。

ここで確認するのは、データとモデルの接続です。

```
```text
x: (1, N, 7) # 座標、surface flag、表面法線
cond: (1, 1, 3) # [Mach, AoA, beta]
direction prompt: (1, N, 4) # [flow_dir_x, flow_dir_y, flow_dir_z, Mach/3]
fx = concat(x, direction): (1, N, 11)
model(xyz, fx) -> pred_norm: (1, N, 4)
```
```

direction prompt は、流入条件を全点に伝えるための特徴です。翼表面の各点は同じMach、AoA、betaのケースに属しているため、この4次元promptはsample内の全点に同じ値として繰り返し付与されます。

学習がうまくいかない場合、lossやoptimizer以前に、このshape設計が破綻していることがあります。そのため、forward passの確認は本番学習前の重要な検査です。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 20

<!-- transolver-learning-note -->

## モデル設計: データがTransolverを通る経路を見る

このセルでは、学習前のモデルにデータを1回通します。目的は精度確認ではなく、入力設計の確認です。

$x[:, :, :3]$  は座標です。x 全体は7次元の点特徴です。torch\_craft\_direction は cond=[Mach, AoA, beta] から [flow\_dir\_x, flow\_dir\_y, flow\_dir\_z, Mach/3] を作ります。これを各点へ繰り返し付与し、元の7次元特徴と連結したものが fx、つまり11次元の点特徴です。

```
```text
fx = [x, y, z, surface_flag, normal_x, normal_y, normal_z,
      flow_dir_x, flow_dir_y, flow_dir_z, Mach/3]
```
```

モデルは座標 xyz と点特徴 fx を受け取り、各点について [Pressure, WSS\_x, WSS\_y, WSS\_z] の4成分を返します。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 7

```
--- GPUが使える場合はcuda、そうでなければCPUを選びます。 ---
device = resolve_device("auto")
model = model.to(device).eval()

--- forward確認用に、1サンプルから少数点だけ抜き出します。 ---
num_forward_points = min(2048, sample["x"].shape[0])
ids = rng.choice(sample["x"].shape[0], size=num_forward_points, replace=False)
x_np = sample["x"][ids]
y_np = sample["y"][ids]
cond_np = sample["cond"]

--- NumPy配列をPyTorch tensorへ変換し、batch次元を追加します。 ---
x = torch.from_numpy(x_np).unsqueeze(0).to(device)
y = torch.from_numpy(y_np).unsqueeze(0).to(device)
cond = torch.from_numpy(cond_np).view(1, 1, -1).to(device)

--- Mach/AoA/betaから、各点へ付与する4次元promptを作ります。 ---
--- 中身は [flow_dir_x, flow_dir_y, flow_dir_z, Mach/3] です。 ---
direction = torch_craft_direction(x[:, :, :3], cond)
--- 7次元の点特徴xと4次元promptを連結し、fun_dim=11の入力にします。 ---
fx = torch.cat([x, direction], dim=-1)

--- 学習ではなくshape確認なので、勾配を保存せずにforwardします。 ---
with torch.no_grad():
 pred_norm = model(x[:, :, :3], fx)

--- 入力と出力のshapeを表示し、モデル接続が正しいか確認します。 ---
print("device:", device)
print("x shape:", tuple(x.shape))
print("direction shape:", tuple(direction.shape))
```

```
print("fx shape:", tuple(fx.shape))
print("prediction shape:", tuple(pred_norm.shape))
print("target shape:", tuple(y.shape))
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 7 output

device: cuda

x shape: (1, 2048, 7)

direction shape: (1, 2048, 4)

fx shape: (1, 2048, 11)

prediction shape: (1, 2048, 4)

target shape: (1, 2048, 4)

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 22

## 8. 学習レシピを読む

ここから、実際の学習条件を整理します。学習レシピとは、モデルをどう初期化し、どのデータをどれだけ使い、どのlossとoptimizerで重みを更新するか、という実験条件のセットです。

CaseBでは、scratch学習として以下を採用します。

|              |                                   |                         |
|--------------|-----------------------------------|-------------------------|
| Item         | Setting                           | 学習上の意味                  |
| ---          | ---                               | ---                     |
| 初期化          | ランダム初期化                           | GeoPT事前学習の効果を使わない基準モデル  |
| optimizer    | AdamW(lr=1e-3, weight_decay=1e-5) | 勾配で重みを更新しつつ過大な重みを抑える    |
| scheduler    | warmup cosine                     | 初期は慎重に、その後なめらかに学習率を下げる  |
| warmup       | 全stepの5%                          | 学習初期の不安定化を抑える           |
| loss         | target正規化後のMSE                    | PressureとWSSのスケール差を緩和する |
| sampling     | epochごとにランダム点サンプリング               | 巨大点群から毎epoch異なる点を見る     |
| point budget | 30,000 points / sample            | GPUメモリと情報量のバランス         |
| epochs       | 200                               | scratchでも十分に収束を見るための回数  |
| AMP          | bfloat16                          | GPUメモリ削減と高速化            |

なぜnormalized MSEを使うか

PressureとWall Shear Stressでは値のスケールが違います。そのままMSEを取ると、値が大きいfieldだけがlossを支配しやすくなります。そこで、trainデータから平均と標準偏差を計算し、正規化空間でMSEを計算します。

なぜ点をサンプリングするか

1サンプルの全点を毎回使うと、GPUメモリと計算時間が大きくなります。そこで各epochで30,000点を選びます。epoch\_random では、epochごとに選ばれる点が変わるため、200epochを通じてモデルはより広い点群分布を見ることになります。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 23

## 9. 小さな学習でパイプラインを検査する

本番CaseBは200epoch、各sample最大30,000点なので、教材中に気軽に何度も回すには重めです。そのため、ここでは任意でtiny splitを作り、1epoch・2,048点でスモーク学習できるようにしています。

このtiny trainingは精度評価用ではありません。目的は、次が壊れていないことを短時間で確認することです。

- train/validation splitを読める。
- 前処理済み .npy を読める。
- Transolverをランダム初期化できる。
- normalized MSEでlossを計算できる。
- optimizer/schedulerで1epoch分の重み更新ができる。
- metrics.json やcheckpointを保存できる。

本番学習へ進む前の「配線確認」と考えてください。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 24

<!-- transolver-learning-note -->

**学習実行: 小さく回して壊れていないことを確認する**

このセルは、本番前のリハーサルです。tiny trainingで良い精度は期待しません。期待するのは、学習スクリプトが最後まで走り、loss計算、backward、checkpoint保存まで通ることです。

実務では、いきなり200epochを走らせる前に、このような小さい実行で環境とデータ処理を検査すると、時間の無駄を減らせます。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 8

```
--- Trueにすると、少数サンプル・少数点で1epochだけ学習確認します。 ---
```

```
RUN_TINY_TRAINING = False
```

```
--- tiny trainingの出力先と、tiny用splitファイルの保存先です。 ---
```

```
TINY_RUN_DIR = REPO_ROOT / "Transolver_Learn" / "runs" / "notebook3_caseb_tiny_smoke"
```

```
TINY_SPLIT_PATH = TINY_RUN_DIR / "split_tiny.json"
```

```
--- デフォルトでは実行しません。必要なときだけこのブロックが動きます。 ---
```

```
if RUN_TINY_TRAINING:
```

```
 TINY_RUN_DIR.mkdir(parents=True, exist_ok=True)
```

```
 # 元のsplitをコピーし、学習確認用に件数を小さくします。
```

```
 tiny_split = dict(split)
```

```
 tiny_split["train"] = split["train"][:3]
```

```
 tiny_split["val"] = split["val"][:2]
```

```
 tiny_split["test"] = split["test"][:2]
```

```
 TINY_SPLIT_PATH.write_text(json.dumps(tiny_split, indent=2))
```

```
本番と同じ学習スクリプトを、軽い条件で呼び出すコマンドを作ります。
```

```
cmd = [
```

```
 str(REPO_ROOT / ".venv" / "bin" / "python"),
```

```
 str(GEOPT_ROOT / "scripts" / "train_wing_geopt.py"),
```

```
 "--case-id", "B_transolver_scratch_matched",
```

```
 "--split-path", str(TINY_SPLIT_PATH),
```

```
 "--preprocessed-dir", str(PREPROCESSED_DIR),
```

```
 "--epochs", "1",
```

```
 "--point-budget", "2048",
```

```
 "--eval-point-budget", "2048",
```

```
 "--scheduler", "warmup_cosine",
```

```
 "--warmup-ratio", "0.05",
```

```
 "--min-lr-scale", "1e-2",
```

```
--loss", "normalized_mse",
--sampling-policy", "epoch_random",
--amp", "--amp-dtype", "bfloat16",
--device", "cuda" if torch.cuda.is_available() else "cpu",
--output-dir", str(TINY_RUN_DIR),
]
print("Running:")
print(" ".join(cmd))
外部プロセスとして学習スクリプトを実行し、標準出力を取得します。
result = subprocess.run(cmd, cwd=REPO_ROOT, text=True, capture_output=True, check=False)
print(result.stdout[-4000:])
if result.returncode != 0:
 print(result.stderr[-4000:])
 raise RuntimeError(f"tiny training failed with returncode={result.returncode}")

実行後に保存されたmetricsを読み、学習処理が完了したことを確認します。
metrics = json.loads((TINY_RUN_DIR / "metrics.json").read_text())
print("Tiny training metrics:")
print(json.dumps(metrics, indent=2))
else:
 print("Tiny training is skipped. Set RUN_TINY_TRAINING = True to run it.")
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 8 output

Tiny training is skipped. Set RUN\_TINY\_TRAINING = True to run it.

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 26

## 10. 本番CaseBは何を実行するのか

本番学習では、train\_wing\_geopt.py にCaseB設定を渡して200epoch学習します。Notebook内のPythonコードからも実行できますが、長時間処理になるため、実務ではログを取りながらターミナルで実行の方が管理しやすいです。

コマンドで特に見るべき引数は次です。

- --case-id B\_transolver\_scratch\_matched: GeoPT重みを使わないscratch Transolver設定を選ぶ。
- --epochs 200: 200回、train splitを繰り返し見る。
- --point-budget 30000: 各sampleから学習に使う点数を30,000点に制限する。
- --eval-point-budget 30000: validation評価も同じ点数で行う。
- --loss normalized\_mse: 目的変数を正規化した空間でMSEを計算する。
- --sampling-policy epoch\_random: epochごとに点のサンプリングを変える。
- --amp --amp-dtype bfloat16: mixed precisionでメモリ使用量を抑える。

本番のCaseBは以下です。

```
```bash
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True /usr/bin/time -v .venv/bin/python EXPERIMENT/GeoPT/scripts/train_wing_geopt.py --case-id B_transolver_scratch_matched --epochs 200 --point-budget 30000 --eval-point-budget 30000 --scheduler warmup_cosine --warmup-ratio 0.05 --min-lr-scale 1e-2 --loss normalized_mse --sampling-policy epoch_random --amp --amp-dtype bfloat16 --device cuda --output-dir EXPERIMENT/GeoPT/runs/B_transolver_scratch_practical_200ep_30k_bf16 2>&1 | tee EXPERIMENT/GeoPT/reports/B_transolver_scratch_practical_200ep_30k_bf16.log
```
```

学習が終わると、主に次のファイルが出力されます。

| File | 役割 |

|---|---|

| best\_model.pt | validationが最も良かったepochの重み |

| model.pt | 最終epochの重み |

| normalization.pt | 出力を物理単位へ戻すための平均・標準偏差 |

| history.json | epochごとのtrain/validation指標 |

| metrics.json | best epoch、best validation、実行時間など |

| config.json | 学習条件とモデル設定 |

評価は以下です。

```
```bash
```

```
.venv/bin/python EXPERIMENT/GeoPT/scripts/evaluate_wing_geopt.py --model-dir EXPERIMENT/GeoPT/runs/B_transol
```

```
ver_scratch_practical_200ep_30k_bf16 --split test --checkpoint-file best_model.pt --point-budget 30000
```

```
--device cuda --output-json EXPERIMENT/GeoPT/reports/B_transolver_scratch_practical_200ep_30k_bf16_test_eva
```

```
l.json
```

```
```
```

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 27

## 11. 学習済みrunを教材として読む

このworkspaceには、CaseBの200epoch学習結果が保存されています。ここでは、再学習する前に、そのrunから「学習すると何が得られるのか」を確認します。

まず見るべき出力は metrics.json と history.json です。

- metrics.json: best validation値、best epoch、実行時間、GPUメモリ使用量などを要約する。
- history.json: 各epochのtrain loss、validation loss、relative L2、learning rateを記録する。

ここで重要なのは、最終epochではなく best validation epoch を使うことです。学習を長く回すと、最後の重みが必ず最良とは限りません。validationで最も良かった時点の best\_model.pt を評価に使います。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 28

<!-- transolver-learning-note -->

### 結果確認: 学習済みrunから何を読むか

このセルでは、保存済みrunのmetricsとhistoryを読みます。学習結果はモデル重みだけではありません。history.json と metrics.json を読むことで、なぜその重みをbest modelとして採用したのかを説明できます。

ここでは、best validation値、best epoch、train/validation曲線の形を確認します。学習者は、単に数値を眺めるのではなく、「このrunは収束しているか」「validationが悪化していないか」を読み取ってください。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 9

```
--- 既存の200epoch CaseB runの保存先を指定します。 ---
existing_caseb = GEOPT_ROOT / "runs" / "B_transolver_scratch_practical_200ep_30k_bf16"
metrics_path = existing_caseb / "metrics.json"
history_path = existing_caseb / "history.json"

--- metricsがあれば、best epochやbest validationなどの要約を表示します。 ---
if metrics_path.exists():
 metrics = json.loads(metrics_path.read_text())
 print("Existing CaseB metrics:")
 print(json.dumps(metrics, indent=2))
else:
 print("Existing CaseB metrics not found:", metrics_path)

--- historyがあれば、epochごとのtrain/validation曲線を描きます。 ---
if history_path.exists():
 history = json.loads(history_path.read_text()).get("history", [])
 if history:
 # plotしやすいように、historyからepochとrelative L2系列を取り出します。
 epochs = [row["epoch"] for row in history]
 train_rel = [row["train_relative_l2"] for row in history]
 val_rel = [row["val_relative_l2"] for row in history]
 plt.figure(figsize=(7, 4))
 plt.plot(epochs, train_rel, label="train relative L2")
 plt.plot(epochs, val_rel, label="val relative L2")
 plt.xlabel("epoch")
 plt.ylabel("relative L2")
 plt.title("Existing CaseB training curve")
 plt.grid(True)
 plt.legend()
 plt.tight_layout()
```

```
plt.show()
```

```
else:
```

```
print("Existing CaseB history not found:", history_path)
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 9 output

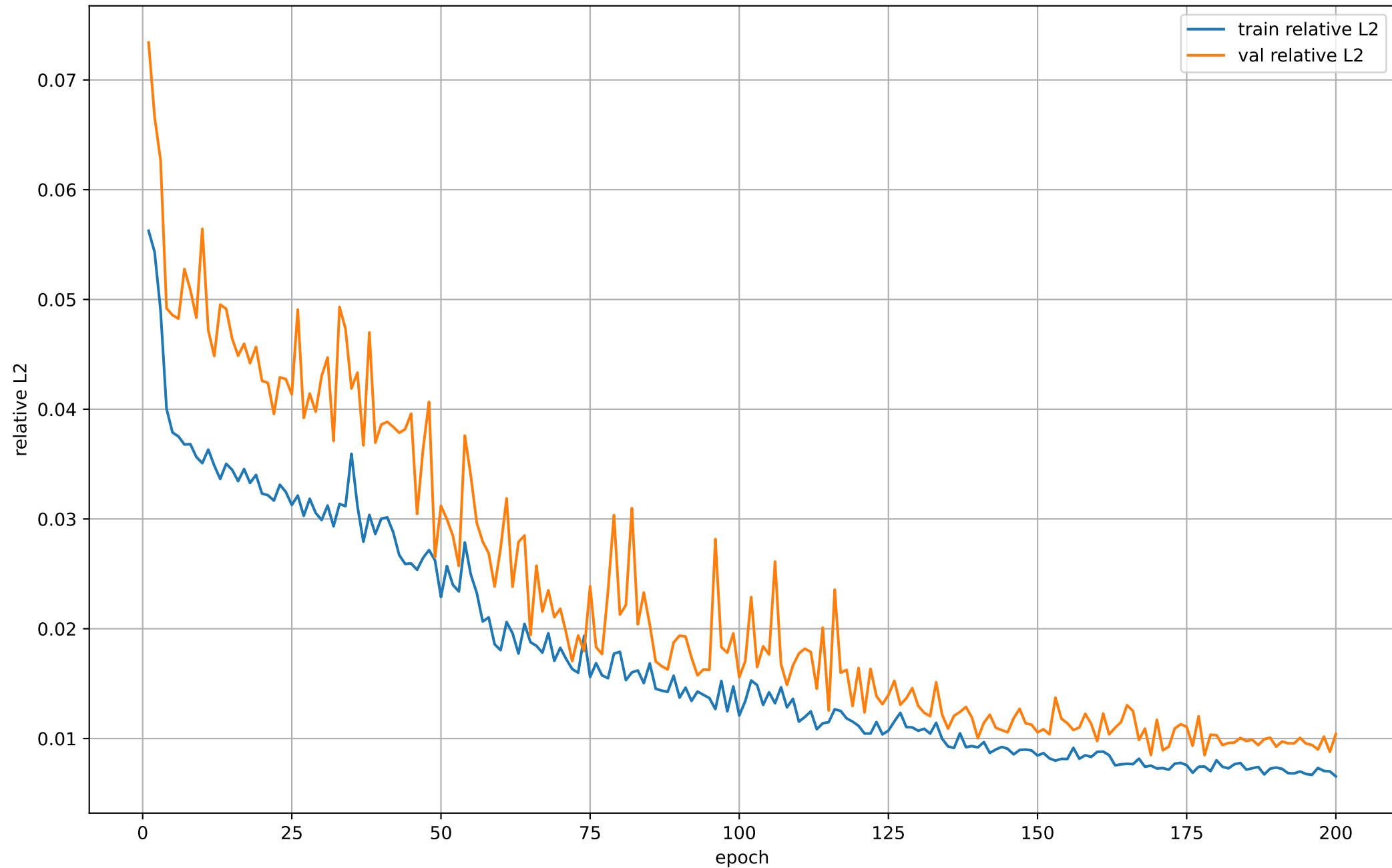
Existing CaseB metrics:

```
{
 "case_id": "B_transolver_scratch_matched",
 "epochs": 200,
 "train_samples": 80,
 "val_samples": 10,
 "best_val_relative_l2": 0.008483962444537679,
 "best_epoch": 178,
 "elapsed_sec": 980.4460657559994,
 "peak_memory_allocated_mb": 2488.92626953125,
 "peak_memory_reserved_mb": 2564.0
}
```

03\_training\_transolver\_shiftwing\_caseb\_ja\_full200\_executed.ipynb:cell29:31: UserWarning: FigureCanvasAgg is not interactive, and thus cannot be shown

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 9 - Figure 1

Existing CaseB training curve



# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 30

## 12. Notebook内で学習を実行する場合

このNotebookでは、学習の全体像を自学できるように、200epoch学習セルも用意しています。ただし、デフォルトでは `RUN_FULL_CASEB_TRAINING = False` にしてあります。これは、Notebookを開いたりPDF化したりするたびに長時間学習が走るのを避けるためです。

実際にこのセルをTrueにすると、次が起こります。

1. CaseB scratch設定でTransolverをランダム初期化する。
2. train splitを使って200epoch学習する。
3. 各epochでvalidationを行い、best validation modelを保持する。
4. history.json、metrics.json、best\_model.ptなどを保存する。

今回の200epoch実行では、学習時間は約17分、best validation relative L2は約 0.00848、best epochは 178 でした。つまり、200epochの後半までvalidation性能が改善し続け、scratch学習として十分に学習が進んだことが分かります。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 31

<!-- transolver-learning-note -->

## 学習実行: 200epoch学習を行うセル

このセルをTrueにすると、本番相当のCaseB学習が走ります。学習中は、各epochでtrain指標とvalidation指標が記録され、validation relative L2が最も良かった重みが best\_model.pt として保存されます。

通常の教材閲覧ではFalseのままにして、保存済みrunを使って結果の読み方に集中します。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 10

```
--- Trueにすると、本番相当の200epoch学習をNotebookから実行します。 ---
RUN_FULL_CASEB_TRAINING = False

--- 200epoch学習結果を保存するrun directoryとlog pathです。 ---
FULL_CASEB_RUN_DIR = REPO_ROOT / "Transolver_Learn" / "runs" / "notebook3_caseb_full_200ep_30k_bf16"
FULL_CASEB_LOG = REPO_ROOT / "Transolver_Learn" / "runs" / "notebook3_caseb_full_200ep_30k_bf16.log"

--- デフォルトではFalseです。教材閲覧時に誤って長時間学習しないためです。 ---
if RUN_FULL_CASEB_TRAINING:
 # CaseB scratch設定を、train_wing_geopt.pyへ渡すコマンドとして組み立てます。
 cmd = [
 str(REPO_ROOT / ".venv" / "bin" / "python"),
 str(GEOPT_ROOT / "scripts" / "train_wing_geopt.py"),
 "--case-id", "B_transolver_scratch_matched",
 "--epochs", "200",
 "--point-budget", "30000",
 "--eval-point-budget", "30000",
 "--scheduler", "warmup_cosine",
 "--warmup-ratio", "0.05",
 "--min-lr-scale", "1e-2",
 "--loss", "normalized_mse",
 "--sampling-policy", "epoch_random",
 "--amp", "--amp-dtype", "bfloat16",
 "--device", "cuda" if torch.cuda.is_available() else "cpu",
 "--output-dir", str(FULL_CASEB_RUN_DIR),
]
 # CUDAメモリ断片化を抑えるため、PyTorchの環境変数を設定します。
 env = os.environ.copy()
 env["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
 print("Running full CaseB training. This may take a while:")
```

```
print(" ".join(cmd))
FULL_CASEB_LOG.parent.mkdir(parents=True, exist_ok=True)
学習ログをファイルへ保存し、後から実行内容を追跡できるようにします。
with FULL_CASEB_LOG.open("w") as log:
 proc = subprocess.run(cmd, cwd=REPO_ROOT, text=True, stdout=log, stderr=subprocess.STDOUT, env=env)
 if proc.returncode != 0:
 raise RuntimeError(f"Full CaseB training failed. See {FULL_CASEB_LOG}")
 print("Full CaseB training finished:", FULL_CASEB_RUN_DIR)
else:
 print("Full CaseB training is skipped. Existing run will be used if available:", FULL_CASEB_RUN_DIR)
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 10 output

Full CaseB training is skipped. Existing run will be used if available: /home/hiroaki-ozasa/Desktop/Developmen

t/PINN\_CFD

/Transolver\_Learn/runs/notebook3\_caseb\_full\_200ep\_30k\_bf16

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 33

### 13. 学習曲線から何を判断するか

学習済みrunの history.json から、epochごとのtrain/validation指標を表示します。この図は、「学習した結果どうなったか」を読むための中心です。

見るポイントは次です。

- train lossが下がる: モデルが学習データを説明できるようになっている。
- validation lossも下がる: 学習データだけでなく、未学習のvalidation形状にも改善が出ている。
- trainだけ下がりvalidationが悪化する: 過学習の可能性はある。
- best epochが後半にある: まだ学習が進んでいた、または200epochが妥当だった可能性がある。
- learning rateがwarmup後に下がる: schedulerが意図通り動いている。

今回のrunではbest epochが178付近で、validation relative L2は約0.00848まで下がりました。これは、scratch学習でもTransolverがShift-WINGの表面物理量をかなりよく近似できていることを示します。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 34

<!-- transolver-learning-note -->

### 結果確認: 学習曲線を読む

このセルの出力は、学習が成功したかを判断する中心です。trainとvalidationが両方下がっていれば、モデルは学習データだけでなく未知validation形状にも対応し始めています。

今回のrunではbest epochが178で、200epochの後半まで改善が続きました。これは、scratch学習として200epoch程度回す意味があったことを示します。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 11

```
--- 以降の評価で使う学習済みrun directoryを指定します。 ---
CASEB_MODEL_DIR = FULL_CASEB_RUN_DIR
history_path = CASEB_MODEL_DIR / "history.json"
metrics_path = CASEB_MODEL_DIR / "metrics.json"

--- 学習履歴がない場合は、評価に進めないため明示的に停止します。 ---
if not history_path.exists():
 raise FileNotFoundError(f"CaseB history not found: {history_path}")

--- 学習履歴と要約metricsを読み込みます。 ---
history = json.loads(history_path.read_text()).get("history", [])
metrics = json.loads(metrics_path.read_text()) if metrics_path.exists() else {}

print("CaseB metrics:")
print(json.dumps(metrics, indent=2))

--- plot用に、epochごとのloss、relative L2、 learning rateを配列化します。 ---
epochs = np.array([row["epoch"] for row in history])
train_loss = np.array([row["train_loss"] for row in history])
val_loss = np.array([row["val_loss"] for row in history])
train_rel = np.array([row["train_relative_l2"] for row in history])
val_rel = np.array([row["val_relative_l2"] for row in history])
lr = np.array([row["lr"] for row in history])
best_epoch = metrics.get("best_epoch")
best_val = metrics.get("best_val_relative_l2")

--- loss、relative L2、 learning rateを横並びで可視化します。 ---
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].plot(epochs, train_loss, label="train loss")
axes[0].plot(epochs, val_loss, label="val loss")
```

```
axes[0].set_title("Normalized MSE loss")
axes[0].set_xlabel("epoch")
axes[0].set_ylabel("loss")
axes[0].grid(True)
axes[0].legend()

axes[1].plot(epochs, train_rel, label="train relative L2")
axes[1].plot(epochs, val_rel, label="val relative L2")
--- validationが最良だったepochを赤点で示します。 ---
if best_epoch is not None:
 axes[1].scatter([best_epoch], [best_val], color="red", zorder=5, label=f"best epoch {best_epoch}")
axes[1].set_title("Relative L2")
axes[1].set_xlabel("epoch")
axes[1].set_ylabel("relative L2")
axes[1].grid(True)
axes[1].legend()

axes[2].plot(epochs, lr, color="tab:green")
axes[2].set_title("Learning rate schedule")
axes[2].set_xlabel("epoch")
axes[2].set_ylabel("lr")
axes[2].grid(True)
plt.tight_layout()
plt.show()
```

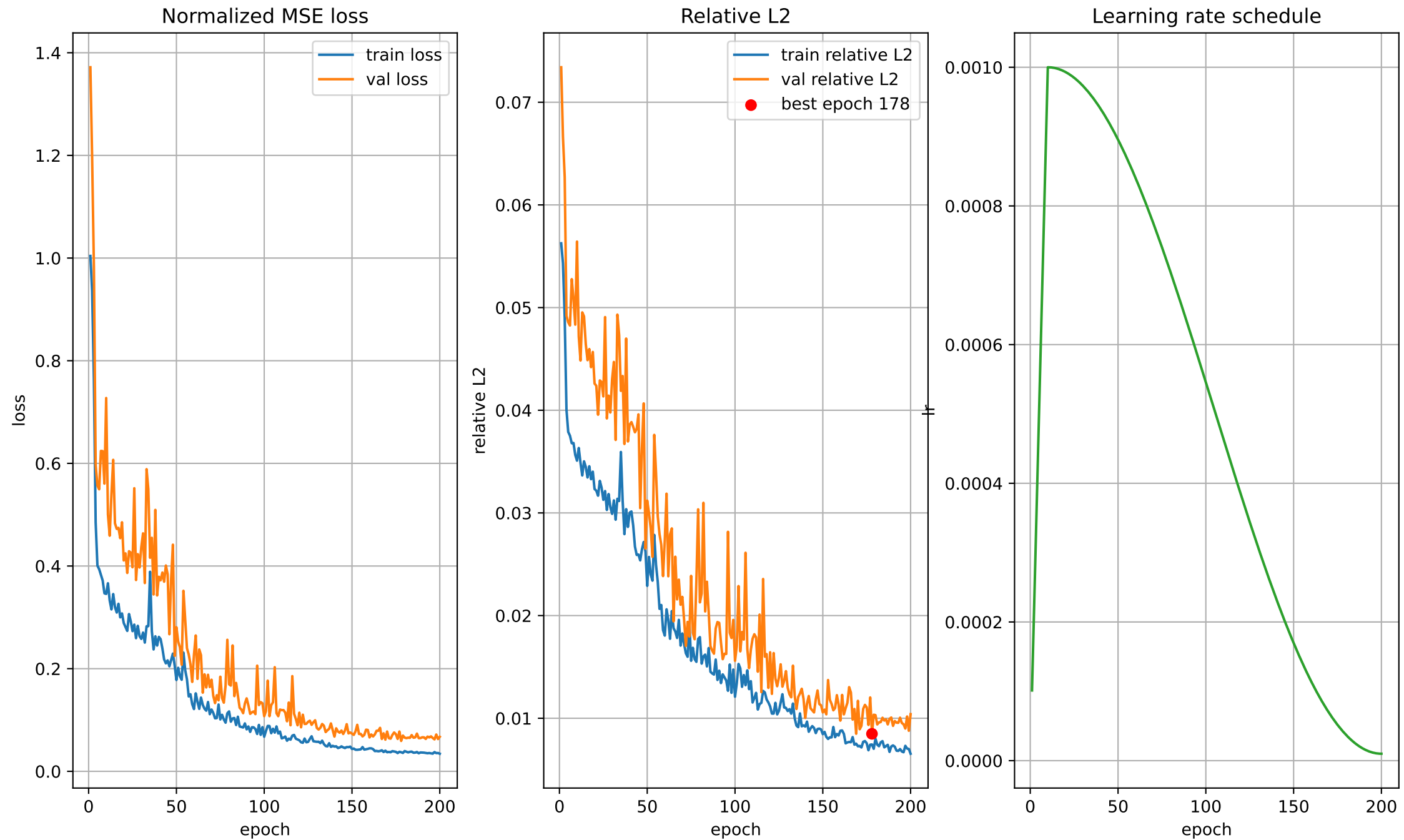
## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 11 output

CaseB metrics:

```
{
 "case_id": "B_transolver_scratch_matched",
 "epochs": 200,
 "train_samples": 80,
 "val_samples": 10,
 "best_val_relative_l2": 0.008483962444537679,
 "best_epoch": 178,
 "elapsed_sec": 1021.4318312541582,
 "peak_memory_allocated_mb": 2488.92626953125,
 "peak_memory_reserved_mb": 2564.0
}
```

03\_training\_transolver\_shiftwing\_caseb\_ja\_full200\_executed.ipynb:cell35:54: UserWarning: FigureCanvasAgg is not interactive, and thus cannot be shown

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 11 - Figure 1



## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 36

<!-- transolver-result-note -->

### 結果の読み取り: 学習は進んだか

この図でvalidation relative L2が下がっていれば、モデルはtrain splitだけでなくvalidation形状にも一般化し始めています。今回のrunではbest validation relative L2が約 0.00848 まで下がり、best epochは 178 でした。

この結果から、少なくとも200epoch付近までは学習を続ける意味があったことが分かります。一方、trainとvalidationの差が大きく開く場合は過学習を疑います。今回の教材では、学習曲線が大きく破綻していないことを確認するのが主目的です。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 37

## 14. test splitで未知形状への性能を見る

validationは学習中のモデル選択に使いました。最終的な性能確認では、学習にもモデル選択にも使っていないtest splitを使います。

ここでは best\_model.pt を読み込み、各test sampleから30,000点をサンプリングして推論します。予測値は正規化空間で出てくるため、normalization.pt に保存された平均・標準偏差で物理単位へ戻します。

この節で見る指標は次です。

- relative\_l2\_all: PressureとWSSをまとめた全体の相対誤差。
- pressure\_relative\_l2: Pressureだけの相対誤差。
- wss\_macro\_relative\_l2: WSS 3成分の平均的な相対誤差。
- pressure\_mae\_pa: Pressure絶対誤差の平均。
- pressure\_rmse\_pa: 大きな誤差をより強く反映するPressure RMSE。
- Pressure誤差percentile: 点ごとの誤差割合を中央値、P75、P90などで見る。

この環境のrunでは、Pressure誤差割合は中央値が約 0.22%、P75が約 0.46%、P90が約 0.98% でした。多くの点では1%未満の誤差に収まっています。一方でP99以降や最大値は大きくなるため、局所的な高誤差領域が残っていることも分かります。

誤差%は以下で定義します。

```
```text
abs(pred_pressure - true_pressure) / max(abs(true_pressure), 1e-12) * 100
```
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 38

<!-- transolver-learning-note -->

### 結果確認: best modelをtest splitで評価する

このセルでは、validationで選ばれたbest modelをtest splitへ適用します。ここで初めて、学習にもモデル選択にも使っていない形状に対する性能を見ます。

推論後には正規化を戻し、Pressureの絶対誤差と誤差割合を計算します。surrogate modelとして使う場合、relative L2だけでなく、実際の物理単位Paでどの程度外れているかも重要です。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 12

```
--- test評価で使うサンプリング関数と誤差指標を読み込みます。 ---
```

```
from wing_geopt_common import sample_point_ids, relative_l2, per_field_relative_l2
```

```
--- 保存済みconfigから同じアーキテクチャを再構築し、best_modelを読み込みます。 ---
```

```
def load_caseb_best_model(model_dir: Path, device: torch.device):
```

```
 # config.jsonには、学習時のモデル設定が保存されています。
```

```
 cfg = json.loads((model_dir / "config.json").read_text())
```

```
 payload = cfg["model_args"]
```

```
 model_args = build_geopt_model_args(
```

```
 out_dim=int(payload["out_dim"]),
```

```
 n_hidden=int(payload["n_hidden"]),
```

```
 n_layers=int(payload["n_layers"]),
```

```
 n_heads=int(payload["n_heads"]),
```

```
 mlp_ratio=int(payload["mlp_ratio"]),
```

```
 slice_num=int(payload["slice_num"]),
```

```
 dropout=float(payload["dropout"]),
```

```
 checkpoint=int(payload["checkpoint"]),
```

```
)
```

```
 model = import_geopt_transolver()(model_args).to(device)
```

```
 # validationで最良だった重みをモデルへ読み込みます。
```

```
 model.load_state_dict(torch.load(model_dir / "best_model.pt", map_location=device))
```

```
 model.eval()
```

```
 # 正規化済み出力を物理単位へ戻すための平均・標準偏差を読み込みます。
```

```
 normalization = {
```

```
 key: value.to(device)
```

```
 for key, value in torch.load(model_dir / "normalization.pt", map_location=device).items()
```

```
 }
```

```
 return model, normalization
```

```

--- 1つのsampleについて、点を選び、推論し、物理単位へ戻す関数です。 ---
def predict_sample(model, normalization, sample_name: str, *, point_budget: int, seed: int, device: torch.device):
 sample = load_preprocessed_sample(PREPROCESSED_DIR, sample_name)
 # 評価点数を揃えるため、指定point_budgetだけ点を選びます。
 ids = sample_point_ids(sample["x"].shape[0], point_budget, seed=seed)
 x_np = sample["x"][ids]
 y_np = sample["y"][ids]
 cond_np = sample["cond"]
 x = torch.from_numpy(x_np).unsqueeze(0).to(device)
 cond = torch.from_numpy(cond_np).view(1, 1, -1).to(device)
 # 評価では勾配不要です。GPUではbfloat16 autocastも使います。
 with torch.no_grad(), torch.autocast(device_type=device.type, dtype=torch.bfloat16, enabled=device.type == "cuda"):
 :
 direction = torch_craft_direction(x[:, :, :3], cond)
 pred_norm = model(x[:, :, :3], torch.cat([x, direction], dim=-1))
 # モデル出力は正規化空間なので、平均と標準偏差で元の単位へ戻します。
 pred = pred_norm.float() * normalization["target_std"] + normalization["target_mean"]
 return ids, x_np, y_np, pred.squeeze(0).detach().cpu().numpy()

device = resolve_device("auto")
caseb_model, caseb_norm = load_caseb_best_model(CASEB_MODEL_DIR, device)

--- test評価で各sampleから使う点数です。本番評価と同じ30,000点にします。 ---
POINT_BUDGET_EVAL = 30000
pressure_abs_errors = []
pressure_percent_errors = []
rows = []

```

```

--- test splitの各sampleを順に推論し、Pressure誤差を集計します。 ---
for index, sample_name in enumerate(split["test"]):
 ids, x_eval, y_true, y_pred = predict_sample(
 caseb_model,
 caseb_norm,
 sample_name,
 point_budget=POINT_BUDGET_EVAL,
 seed=10000 + index,
 device=device,
)
 error = y_pred - y_true
 pressure_error = error[:, 0]
 pressure_abs = np.abs(pressure_error)
 # 誤差割合[%]は、Pressure絶対値に対する相対的な外れ幅として計算します。
 pressure_pct = pressure_abs / np.maximum(np.abs(y_true[:, 0]), 1e-12) * 100.0
 pressure_abs_errors.append(pressure_abs)
 pressure_percent_errors.append(pressure_pct)
 per_field = per_field_relative_l2(y_pred, y_true)
 rows.append({
 "sample": sample_name,
 "points": int(y_true.shape[0]),
 "relative_l2_all": relative_l2(y_pred, y_true),
 "pressure_relative_l2": per_field[0],
 "wss_macro_relative_l2": float(np.mean(per_field[1:])),
 "pressure_mae_pa": float(np.mean(pressure_abs)),
 "pressure_rmse_pa": float(np.sqrt(np.mean(pressure_error ** 2))),
 })

all_abs = np.concatenate(pressure_abs_errors)
all_pct = np.concatenate(pressure_percent_errors)

--- sampleごとの指標と、全点をまとめたPressure誤差percentileを表示します。 ---

```

```
print("Sample-level test metrics:")
for row in rows:
 print(row)

print("\nPressure error percentiles over sampled test points:")
for p in [50, 75, 90, 95, 99, 99.9, 100]:
 print(f"P{p}>5g: abs={np.percentile(all_abs, p):.4f} Pa, percent={np.percentile(all_pct, p):.6f}%")
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 12 output

Sample-level test metrics:

```
{'sample': 'sample_000001', 'points': 30000, 'relative_l2_all': 0.006029002197526745, 'pressure_relative_l2':
0.00602875
3908765684, 'wss_macro_relative_l2': 0.12733230196959386, 'pressure_mae_pa': 80.86441802978516, 'pressure_rmse
_pa': 140.
7596893310547}
{'sample': 'sample_000005', 'points': 30000, 'relative_l2_all': 0.007148254296488571, 'pressure_relative_l2':
0.00714807
4900945944, 'wss_macro_relative_l2': 0.13898902306982272, 'pressure_mae_pa': 96.19056701660156, 'pressure_rmse
_pa': 167.
7848358154297}
{'sample': 'sample_000010', 'points': 30000, 'relative_l2_all': 0.006062641710496518, 'pressure_relative_l2':
0.00606244
35065864, 'wss_macro_relative_l2': 0.14316831930689475, 'pressure_mae_pa': 83.91063690185547, 'pressure_rmse_p
a': 142.50
970458984375}
{'sample': 'sample_000029', 'points': 30000, 'relative_l2_all': 0.00892554866357965, 'pressure_relative_l2': 0
.008925380
575543932, 'wss_macro_relative_l2': 0.1472299423608289, 'pressure_mae_pa': 104.09197998046875, 'pressure_rmse_
pa': 209.3
039093017578}
{'sample': 'sample_000043', 'points': 30000, 'relative_l2_all': 0.0051766223183399756, 'pressure_relative_l2':
0.0051763
90712498569, 'wss_macro_relative_l2': 0.12634596531276865, 'pressure_mae_pa': 72.00606536865234, 'pressure_rms
e_pa': 121
.50475311279297}
{'sample': 'sample_000047', 'points': 30000, 'relative_l2_all': 0.010036917799404459, 'pressure_relative_l2':
0.01003675
8140390628, 'wss_macro_relative_l2': 0.157950326179015, 'pressure_mae_pa': 117.06288146972656, 'pressure_rmse_
pa': 235.2
```

```
1588134765625}
{'sample': 'sample_000066', 'points': 30000, 'relative_l2_all': 0.013088001687214018, 'pressure_relative_l2':
0.01308779
3217469671, 'wss_macro_relative_l2': 0.18114517307384093, 'pressure_mae_pa': 139.1939239501953, 'pressure_rmse
_pa': 304.
6949157714844}
{'sample': 'sample_000069', 'points': 30000, 'relative_l2_all': 0.014217798281231979, 'pressure_relative_l2':
0.01421756
306704024, 'wss_macro_relative_l2': 0.18700531026643796, 'pressure_mae_pa': 142.2437286376953, 'pressure_rmse_
pa': 330.9
676513671875}
{'sample': 'sample_000073', 'points': 30000, 'relative_l2_all': 0.006927125942550342, 'pressure_relative_l2':
0.00692693
6776815897, 'wss_macro_relative_l2': 0.15620321010540839, 'pressure_mae_pa': 88.54916381835938, 'pressure_rmse
_pa': 162.
87278747558594}
{'sample': 'sample_000095', 'points': 30000, 'relative_l2_all': 0.008934457735705852, 'pressure_relative_l2':
0.00893425
963746597, 'wss_macro_relative_l2': 0.14881489257465094, 'pressure_mae_pa': 110.49606323242188, 'pressure_rmse
_pa': 208.
81346130371094}
```

Pressure error percentiles over sampled test points:

```
P 50: abs=52.1025 Pa, percent=0.220966%
P 75: abs=108.8892 Pa, percent=0.462032%
P 90: abs=227.1291 Pa, percent=0.984134%
P 95: abs=358.5247 Pa, percent=1.584222%
P 99: abs=916.7975 Pa, percent=4.684409%
P 99.9: abs=2101.0149 Pa, percent=15.346943%
P 100: abs=5448.2280 Pa, percent=73.966393%
```

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 40

<!-- transolver-result-note -->

## 結果の読み取り: Pressure誤差をpercentileで見る

出力されるPressure誤差percentileは、点ごとの誤差割合の分布です。中央値は「典型的な点での誤差」、P90は「90%の点がこの誤差以下」という意味です。

このrunでは、Pressure誤差割合の中央値が約 0.22%、P75が約 0.46%、P90が約 0.98% でした。多くの点では良好ですが、P99以降や最大値は大きくなります。これは、全体としてはよく予測できても、局的に難しい領域が残っていることを示します。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 41

<!-- transolver-learning-note -->

## 結果確認: 誤差の分布を見る

このセルでは、全test点のPressure誤差を分布として見ます。平均値だけでは、一部の大きな誤差が隠れることがあります。中央値、P75、P90、P99などのpercentileとヒストグラムを組み合わせると、多くの点での典型誤差と、局所的な外れ誤差の両方を把握できます。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 13

```
--- Pressure誤差を、絶対誤差Paと誤差割合%の2種類で可視化します。 ---
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

--- 左図: 物理単位Paでどれくらい外れたかを表示します。 ---
axes[0].hist(all_abs, bins=80, color="tab:blue", alpha=0.85)
axes[0].set_title("Pressure absolute error distribution")
axes[0].set_xlabel("abs error [Pa]")
axes[0].set_ylabel("count")
axes[0].grid(True)

極端な外れ値で図が潰れないように、P99.5までを主表示にする。
--- 右図: 外れ値で図が潰れないよう、P99.5で表示範囲を制限します。 ---
pct_clip = np.percentile(all_pct, 99.5)
axes[1].hist(np.clip(all_pct, 0, pct_clip), bins=80, color="tab:orange", alpha=0.85)
axes[1].set_title("Pressure percent error distribution (clipped at P99.5)")
axes[1].set_xlabel("error [% of truth]")
axes[1].set_ylabel("count")
axes[1].grid(True)

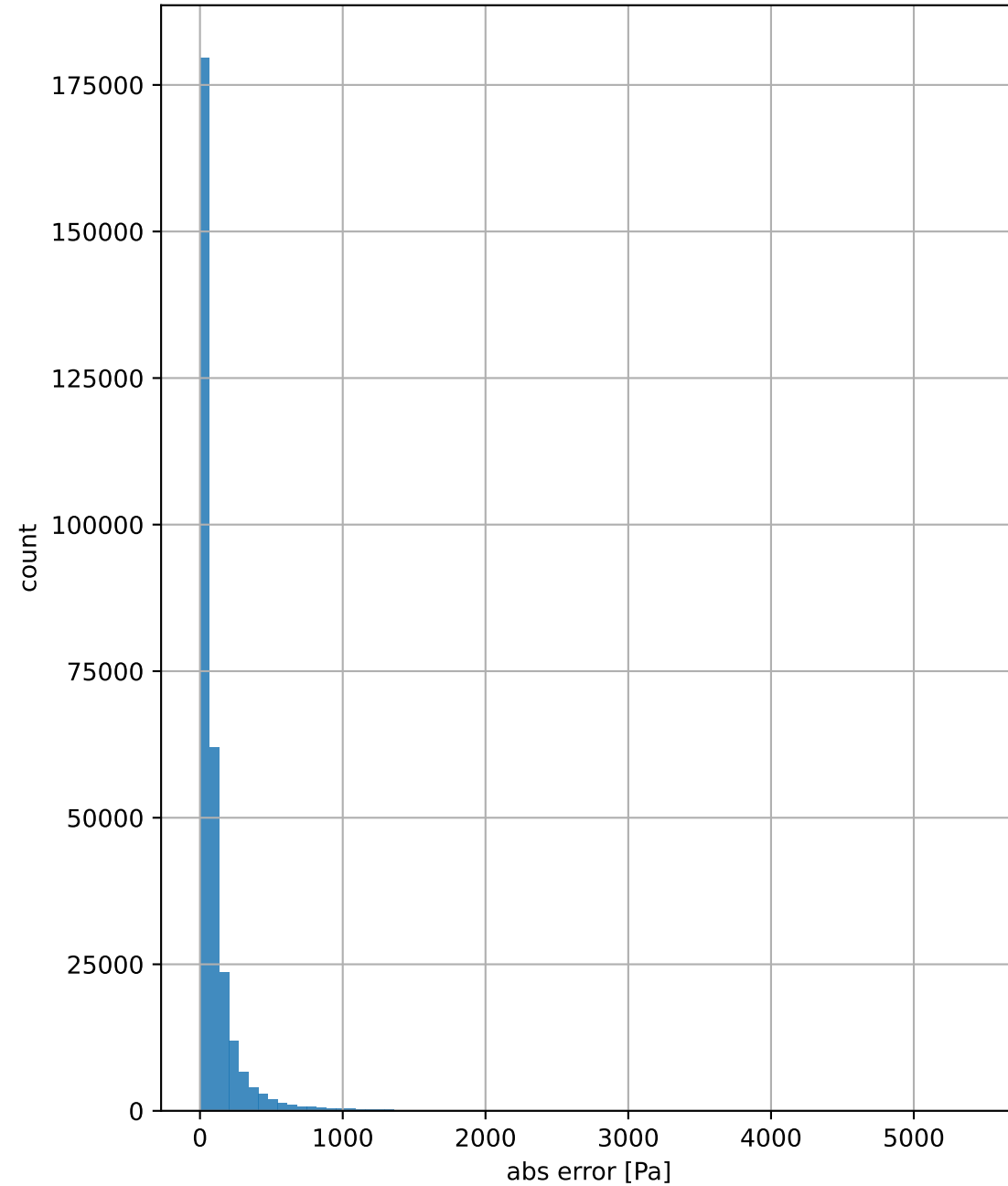
plt.tight_layout()
plt.show()
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 13 output

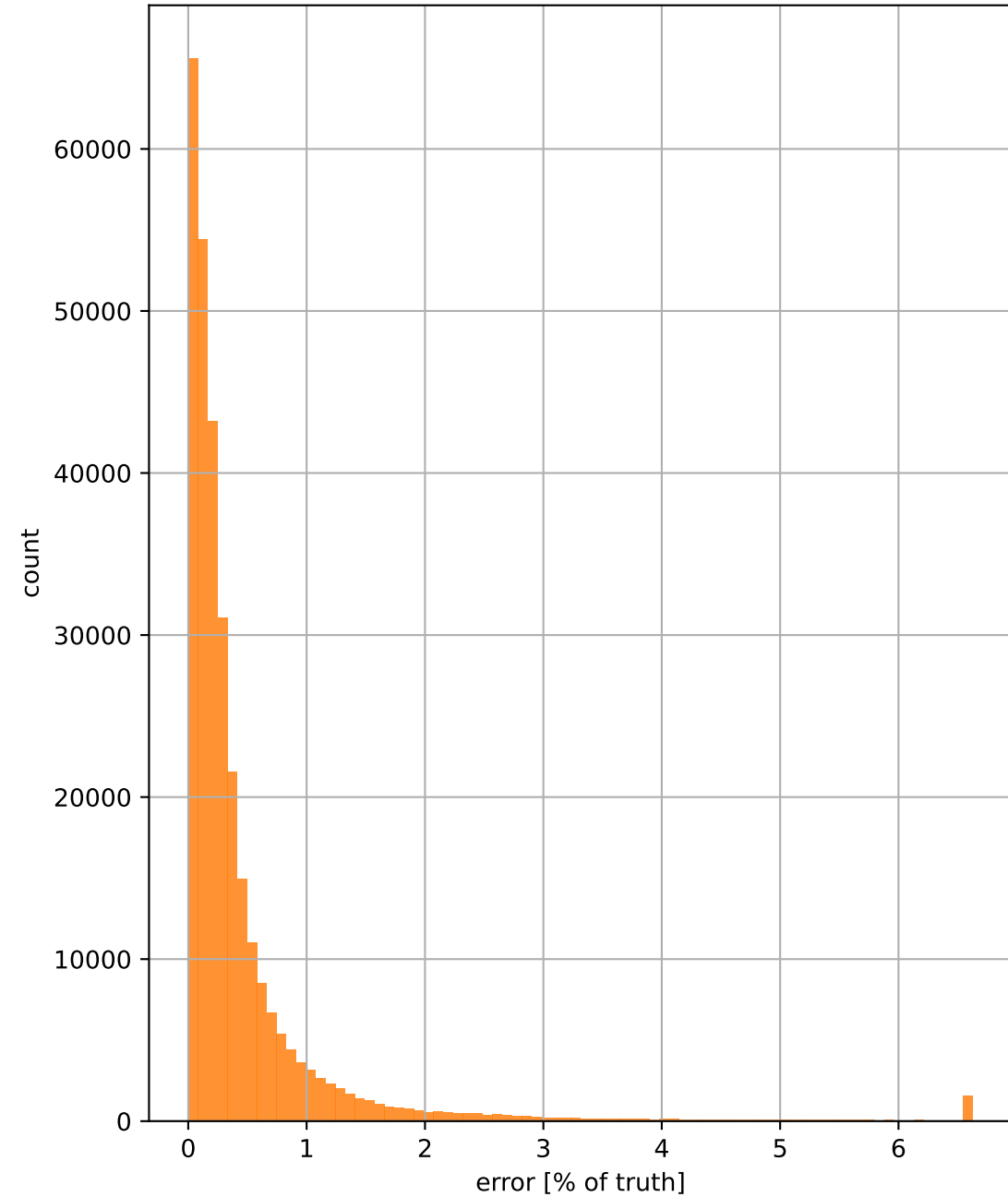
03\_training\_transolver\_shiftwing\_caseb\_ja\_full200\_executed.ipynb:cell42:21: UserWarning: FigureCanvasAgg is not interactive, and thus cannot be shown

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 13 - Figure 1

Pressure absolute error distribution



Pressure percent error distribution (clipped at P99.5)



## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 43

<!-- transolver-result-note -->

### 結果の読み取り: 分布の裾に注意する

左図の絶対誤差は、Pa単位でどれくらい外れているかを示します。右図の誤差%は、正解Pressureの大きさに対してどれくらい外れているかを示します。

誤差%は、正解値が非常に小さい点では大きく見えやすいので、P99.5でclipした図を使っています。平均値だけで判断せず、分布の中心と裾を分けて読むことが重要です。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 44

## 15. 誤差がどこに出ているかを見る

数値指標だけでは、モデルがどの場所で外しているのか分かりません。そこで代表test sampleについて、正解Pressure、予測Pressure、絶対誤差、誤差%を同じ点群上に並べて見ます。

この図で学習者が見るべきことは次です。

- 正解と予測のPressure分布が全体として似ているか。
- 高圧・低圧の位置がずれていないか。
- 誤差が翼面全体に広がっているのか、一部領域に集中しているのか。
- 誤差%が大きい領域が、物理量の絶対値が小さい場所に集中していないか。

今回のようなsurrogate modelでは、平均誤差が小さくても、前縁、後縁、急な圧力勾配、局所的な剥離に近い領域で誤差が残ることがあります。数値指標と空間分布をセットで見ることで、次にデータ追加、サンプリング強化、モデル容量変更のどれを検討すべきか判断しやすくなります。

ここではNotebook/PDFで見やすくするため、VTPの3Dレンダリングではなく、正規化済み座標の x-y 平面へscatter表示します。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 45

<!-- transolver-learning-note -->

## 結果確認: 誤差の場所を見る

このセルでは、誤差を空間分布として見ます。誤差が翼面全体に均一に出ているのか、前縁や後縁など特定領域に集中しているのかで、次を取る改善策が変わります。

この図は、モデル改善のためだけでなく、予測結果をCFD利用者へ説明するためにも重要です。

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 14

```
--- test splitから代表sampleを1つ選び、空間分布を詳しく見ます。 ---
CONTOUR_SAMPLE = split["test"][0]
CONTOUR_POINTS = 60000

--- 代表sampleを推論し、座標、正解、予測を取得します。 ---
ids, x_plot, y_true_plot, y_pred_plot = predict_sample(
 caseb_model,
 caseb_norm,
 CONTOUR_SAMPLE,
 point_budget=CONTOUR_POINTS,
 seed=2026,
 device=device,
)

--- Pressure成分だけを取り出し、絶対誤差と誤差割合を計算します。 ---
truth_p = y_true_plot[:, 0]
pred_p = y_pred_plot[:, 0]
abs_err_p = np.abs(pred_p - truth_p)
pct_err_p = abs_err_p / np.maximum(np.abs(truth_p), 1e-12) * 100.0

--- 正解と予測を同じカラスケールで比較できるよう、表示範囲を揃えます。 ---
pmin = float(min(np.percentile(truth_p, 1), np.percentile(pred_p, 1)))
pmax = float(max(np.percentile(truth_p, 99), np.percentile(pred_p, 99)))
err_max = float(np.percentile(abs_err_p, 99))

--- 正解、予測、絶対誤差、誤差%を横並びで表示します。 ---
fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))
scatter_kwargs = dict(s=1, alpha=0.85)

im0 = axes[0].scatter(x_plot[:, 0], x_plot[:, 1], c=truth_p, cmap="turbo", vmin=pmin, vmax=pmax, **scatter_kwargs)
```

```

axes[0].set_title("Truth pressure")
axes[0].set_aspect("equal", adjustable="box")
plt.colorbar(im0, ax=axes[0], fraction=0.046, pad=0.04)

im1 = axes[1].scatter(x_plot[:, 0], x_plot[:, 1], c=pred_p, cmap="turbo", vmin=pmin, vmax=pmax, **scatter_kwar
gs)
axes[1].set_title("Predicted pressure")
axes[1].set_aspect("equal", adjustable="box")
plt.colorbar(im1, ax=axes[1], fraction=0.046, pad=0.04)

im2 = axes[2].scatter(x_plot[:, 0], x_plot[:, 1], c=abs_err_p, cmap="magma", vmin=0, vmax=err_max, **scatter_k
wargs)
axes[2].set_title("Abs error [Pa]")
axes[2].set_aspect("equal", adjustable="box")
plt.colorbar(im2, ax=axes[2], fraction=0.046, pad=0.04)

pct_vmax = float(np.percentile(pct_err_p, 99))
im3 = axes[3].scatter(x_plot[:, 0], x_plot[:, 1], c=np.clip(pct_err_p, 0, pct_vmax), cmap="viridis", vmin=0, v
max=pct_
vmax, **scatter_kwargs)
axes[3].set_title("Percent error [%], clipped P99")
axes[3].set_aspect("equal", adjustable="box")
plt.colorbar(im3, ax=axes[3], fraction=0.046, pad=0.04)

--- 全subplotに軸ラベルを付け、座標の見方を揃えます。 ---
for ax in axes:
 ax.set_xlabel("normalized x")
 ax.set_ylabel("normalized y")

fig.suptitle(f"CaseB best model on {CONTOUR_SAMPLE}")
plt.tight_layout()
plt.show()

```

```
--- 図に加えて、代表sampleの数値指標も表示します。 ---
print("Contour sample:", CONTOUR_SAMPLE)
print("pressure relative L2:", relative_l2(pred_p.reshape(-1, 1), truth_p.reshape(-1, 1)))
print("MAE [Pa]:", float(np.mean(abs_err_p)))
print("RMSE [Pa]:", float(np.sqrt(np.mean((pred_p - truth_p) ** 2))))
print("median percent error [%]:", float(np.percentile(pct_err_p, 50)))
print("P90 percent error [%]:", float(np.percentile(pct_err_p, 90)))
print("P99 percent error [%]:", float(np.percentile(pct_err_p, 99)))
```

## Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 14 output

03\_training\_transolver\_shiftwing\_caseb\_ja\_full200\_executed.ipynb:cell46:57: UserWarning: FigureCanvasAgg is no

n-interactive

ive, and thus cannot be shown

Contour sample: sample\_000001

pressure relative L2: 0.011679591453228302

MAE [Pa]: 131.1541290283203

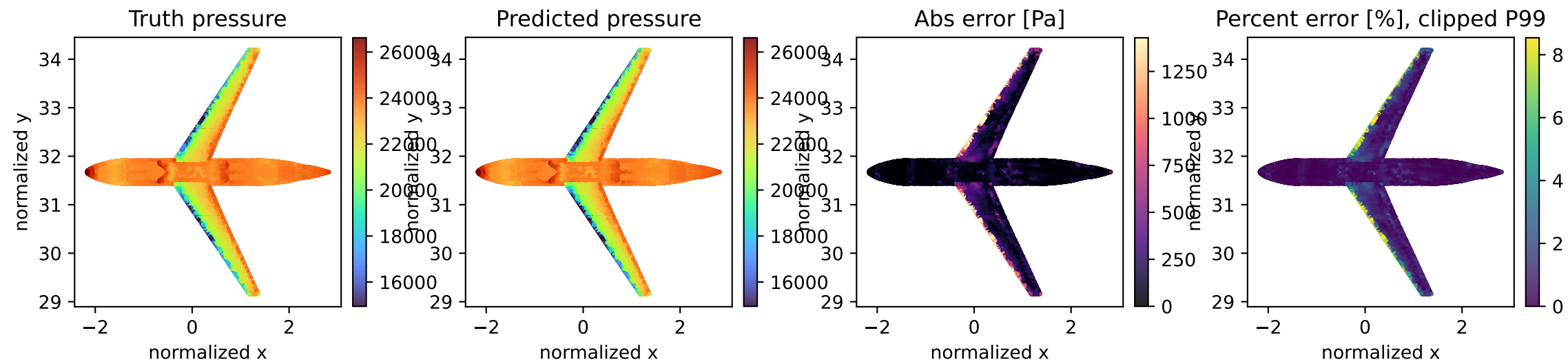
RMSE [Pa]: 272.8179931640625

median percent error [%]: 0.24678783118724823

P90 percent error [%]: 1.290942907333374

P99 percent error [%]: 8.538398742675781

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Code 14 - Figure 1



# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 47

<!-- transolver-result-note -->

## 結果の読み取り: 次に改善するならどこか

正解と予測の色分布が似ていれば、モデルは大域的なPressure構造を捉えています。誤差図で高誤差が局所的に出る場合、その場所がモデルにとって難しい領域です。

改善を検討するなら、まずこの高誤差領域が前縁、後縁、低Pressure絶対値領域、点密度の粗い領域のどれに対応するかを確認します。その上で、サンプリング、データ追加、loss重み、モデル容量などを検討します。

# Notebook 3: Shift-WING CaseB 200epoch実行結果 / Markdown 48

## まとめ

このNotebookでは、Shift-WINGデータを使ってTransolver surrogate modelをスクラッチ学習する流れを確認しました。

重要なのは、コードを順に実行することではなく、次の因果関係を理解することです。

1. CFD結果を点群テンソル  $x$ 、教師値  $y$ 、条件  $cond$  に整理する。
2. train / validation / testを分け、学習、モデル選択、最終評価の役割を分離する。
3. Transolverでは、全点Attentionではなくphysics sliceを使って大規模点群を扱う。
4. CaseBではGeoPT事前学習重みを使わず、同じTransolver構造をランダム初期化から学習する。
5. normalized\_mse により、PressureとWSSのスケール差を抑えて学習する。
6. 200epoch学習では、best validation relative L2が約 0.00848 まで下がった。
7. test splitのPressure誤差は中央値約 0.22%、P90約 0.98% で、多くの点では良好に予測できている。
8. 一方で、P99以降や局所誤差を見ると、高誤差領域は残るため、空間分布での確認が必要である。

このNotebookを理解できれば、別の形状データ、例えばwind turbineのSTL/VTPを使う場合でも、同じ考え方で「入力形式を作る、Transolverへ入れる、学習曲線を見る、testで誤差分布を見る」という流れを設計できます。